



GrowTogether: A Collaborative Productivity App for Gamified Focus and Shared Accountability

Team Name: Growers

Darlyn Antoinette Baybayon
Jade Marco Morillo
Jan Edward Macagba
Jhon Khriel Cymon Empeno
John Carlos Quintero
Kyte Daiter Percia
Lindsay Faith Bazar
Romand Lansangan

2025/11/29

1. Project Overview.....	3
1.1 Problem Statement.....	3
1.2 Solution Description.....	3
1.3 Core Mechanics.....	4
1.4 Tech Stack Summary.....	4
1.5 Architecture Summary.....	5
2. Frontend.....	5
2.1 Technology Stack and Dependencies.....	5
2.2 Application Architecture.....	6
2.2.1 Folder/file structure overview.....	6
2.2.2 State management approach.....	7
2.3 UI/UX Design.....	7
2.4 Key Screens/Pages.....	8
2.5 API Integration.....	8
2.6 Static Website (Admin Panel).....	8
3. Database.....	8
3.1 Database Architecture Overview.....	8
3.2 Table Schemas.....	9
3.3 Data Relationships.....	16
3.4 Access Patterns.....	16
4. Backend.....	17
4.1 Cloud Architecture Overview.....	17
4.2 Dependencies.....	18
4.3 Environment Configuration.....	18
4.4 Lambda Functions.....	19
4.4.1 create_session.....	19
4.4.2 start_session.....	20
4.4.3 update_session.....	21
4.4.4 end_session.....	22
4.4.5 sendPing_Email.....	23
4.4.6 createUser.....	24
4.4.7 linkBuddy.....	25
4.4.8 login.....	26
4.4.9 updateStreak.....	27
4.4.10 getForestState.....	28
4.4.11 get_buddy_info.....	29
4.5 Other Details.....	31
4.5.1 Authentication for API Calls.....	31
4.5.2 IAM Permissions Required.....	31
4.5.3 Rate Limits.....	31

4.5.4 Business Logic Rules.....	32
4.5.5 Dependencies Between Functions.....	32
5. Project References.....	33
5.1 Flutter Documentation.....	33
5.2 AWS Documentation.....	33
5.3 Glossary of Terms.....	33
6. Appendices.....	35
6.1 Role Descriptions.....	35
6.2 Roles & Contributions.....	35
6.3 Github Link.....	36

1.1 Problem Statement

In both contemporary environments of academic and professional, many individuals often struggle to maintain consistent productivity and healthy work habits. This may be due to digital distractions, lack of structured peer accountability, or fluctuating motivation levels. Even though in today's digital era that many exist productivity tools designed for **personal or individual tracking**, however, there is still a failure to incorporate **collaborative or social reinforcement** that can enhance long-term engagement and behavioral consistency. Studies in collaborative learning and study partnerships have shown that accountability-based peer systems can significantly improve task adherence, engagement, and even overall performance (Jones et al., 2025; Arya et al., 2021).

Furthermore, research in technology-supported learning, in which the current era of education, and behavioral motivation indicate that gamified systems and progress visualizations can strengthen user commitment by transforming abstract goals into visible rewarding progress (Arya et al., 2021). This explains the popularity of digital trackers and spreadsheets shared online for personal productivity like Google Sheets or Notion. However, the absence of applications combining **collaborative accountability with gamified visualization** highlights an opportunity to design a productivity tool that not only promotes individual growth but also fosters shared responsibility and motivation between paired users.

1.2 Solution Description

GrowTogether essentially turns the journey of solitary healthy habit building into a more cooperative game where both players win by staying consistent.

The **Buddy System** will be useful in terms of linking two people to become responsible for contributing to a common outcome, that is growing their forest. Such dynamics will not only develop one's personal growth but will also be indirectly contributing to the development of those outside of themselves, making it a journey of more than one individual. The following are the specific features under this system:

- Two users link via invite code and become accountability partners
- Light social connection through "ping" notifications (no distracting chat)

Besides invite codes, the system can also evolve to include intelligent matchmaking based on user profiles (goals, schedules, learning styles) to pair compatible accountability partners. A lightweight public forum could also allow users to find accountability partners, share milestone achievements, forest visualizations, and success strategies.

Gamified Progress Visualization will make habit building more engaging, immersive, and interactive. Many people lose their way because of their progress and will eventually feel overwhelmed and revert back to their original habits because they only visualize their ideal outcome in their mind. With a gamified progress visualization, the progress becomes more concrete and observable. This will make the progress of the users more apparent in terms of visibility--for what can be seen cannot be forgotten--what cannot be forgotten turns to commitment. The exact features are as follows:

- Shared virtual forest that grows based on combined focus hours
- Visual progression: Seedling -> Tree -> Forest

- Each focus session adds XP and grows trees
- Daily streaks maintain forest health

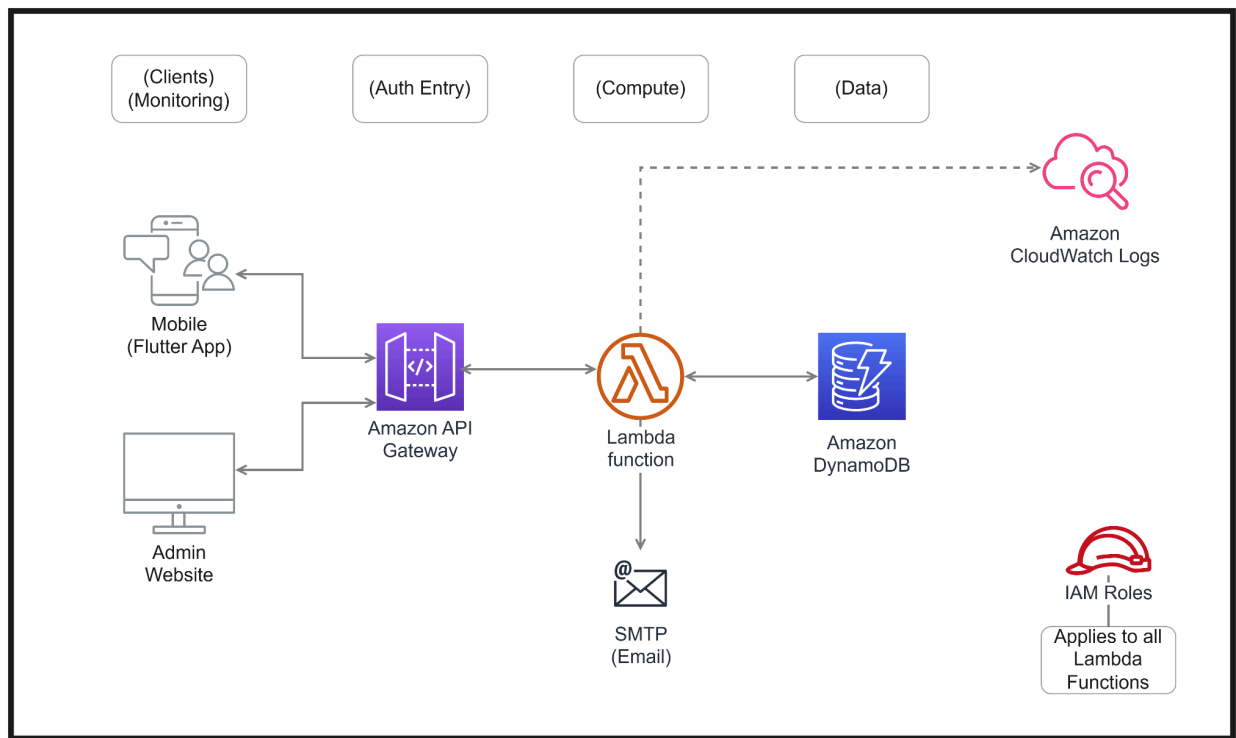
1.3 Core Mechanics

- Focus Timer: Pomodoro-style sessions that log completed work
- Mutual Transparency: See partner's status, daily hours, and streak
- Positive Reinforcement: XP system, streak bonuses (+10% XP per 5-day streak), badges
- Gentle Nudges: Ping feature with 2-hour cooldown to encourage without nagging

1.4 Tech Stack Summary

- Front-End (Flutter)
 - Flutter Packages
 - Http - usage of functions and classes to send API requests to the API gateway
 - Fl Chart - a Flutter library that supports different kinds of charts
- Back-End (AWS)
 - AWS Lambda (Python or Node.js)
 - To execute all business logic (XP, streak, tree growth, analytics calculation).
 - AWS DynamoDB
 - NoSQL database service for storage of all the application's structured data (users, sessions, forest)
 - AWS API Gateway
 - Application's "front door" to access backend services like Lambda. Handles security, rate limiting, and mapping incoming requests to the appropriate Lambda function.
 - AWS SNS
 - AWS notification service that manages the delivery or sending of messages to subscribing endpoints. Essential for sending "pings" to partners.
 - AWS SAM/Serverless Framework
 - Define, deploy, and manage serverless applications consisting of Lambda, API Gateway, and DynamoDB.
 - AWS Cognito
 - Simplifies authorization logic in API Gateway and Lambda (for buddy linking).
- Github

1.5 Architecture Summary



2. Frontend

2.1 Technology Stack and Dependencies

Dart SDK Version 3.9.0 was used together with **Flutter Version 3.38.0** to implement the frontend of the application. The following are packages used:

Package	Version	Purpose
http	1.5.0	Sends API requests to AWS API Gateway for backend communication
provider	6.1.5+1	State management across the application
fl_chart	0.69.0	Data visualization for progress charts (daily, weekly, monthly, XP distribution)
flutter_spinbox	0.13.1	Numeric input controls for timer/session settings
flutter_multi_formatter	2.13.10	Input formatting for forms (phone, currency, etc.)

uuid	4.5.1	Generates unique identifiers for users and sessions
cupertino_icons	1.0.8	iOS-style icons for cross-platform consistency
flutter_lints	5.0.0	Code quality and linting rules (dev dependency)

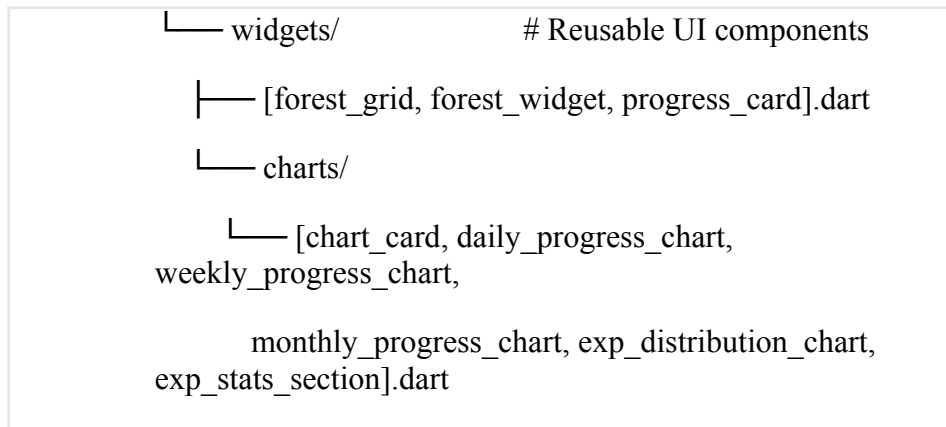
2.2 Application Architecture

2.2.1 Folder/file structure overview

```

lib/
├── main.dart
├── data/                # Static data and configurations
│   ├── [awards_data, awards_by_categories,
│   award_categories].dart
├── login/              # Authentication screens
│   ├── [login_screen, signup_screen].dart
├── models/             # Data models (Plain Dart
Objects)
│   ├── [user, award, daily_progress, weekly_progress,
monthly_progress,
│   exp_distribution, exp_stats].dart
├── screens/            # Main application screens
│   ├── [home_screen, focus_screen, awards_screen,
profile_screen].dart
├── seervices/          # Business logic and API services
│   ├── [progress_service.dart, auth_service.dart,
level_service.dart, user_service.dart]
├── theme/              # Theming configuration
│   ├── [theme_color, light_theme, dark_theme,
timercolor].dart

```



2.2.2 State management approach

The application uses a **hybrid state management approach**:

Approach	Where Used	Description
StatefulWidget (setState)	main.dart, focus_screen.dart, login_screen.dart	Local component state for UI interactions like theme toggling, timer countdown, form inputs, and navigation index
Provider	App-wide state (available but minimally used in current codebase)	Dependency included for scalable state management; can be extended for user session, forest state, and buddy data
Lifting State Up	main.dart	Authentication state (loginUser), theme mode, and page index are managed in the root widget and passed down to child screens

2.3 UI/UX Design

The User Interface and User Experience or UI/UX design of the application is anchored in a gamified, encouraging, and collaborative design philosophy. Upon opening the application, users are immediately presented with their “forest,” a visual representation of their progress and achievements. This approach is intended to cultivate motivation by allowing users to observe their development in a tangible and visually engaging manner. Collaboration is further emphasized through features such as shared progress charts, which display both the user’s and their partner’s data. Additionally, the interface adopts elements commonly found in mobile gaming environments to enhance user engagement and encourage sustained interaction with the platform.

The color palette and thematic direction of the application are guided by a forest motif. The dominant use of green and brown, colors traditionally associated with nature and growth, creates a consistent visual identity aligned with the application's objectives. Nature-inspired elements, including trees, fruits, and related icons, are integrated throughout the interface to reinforce this motif. The awards, achievement tiers, and progression labels are likewise named after botany-related objects to maintain thematic coherence.

Regarding typography, the application employs TT Hoves and TT Hoves Bold. Given that the interface incorporates numerous visually detailed components, the use of clean and minimalist typefaces ensures readability and prevents unnecessary visual complexity. These typographic choices provide a balanced contrast to the richer graphic elements and contribute to a clear, user-friendly interface.

2.4 Key Screens/Pages

- Document each major screen.
- Login/Registration
- Dashboard/Home (forest visualization)
- Focus Timer
- Awards
- Profile/Buddy Connection

2.5 API Integration

- How the app communicates with API Gateway
- Error handling approach

2.6 Static Website (Admin Panel)

- Purpose and features
- Design consistency with mobile app

3. Database

3.1 Database Architecture Overview

The group's application named **GROWTOGETHER** using AWS DynamoDB and not SQL databases, as this feature of AWS provides and shows high scalability and flexible schema structures, which is ideal for the group project as the app handles real-time activity and handles user data. Again, the feature provides the platform (**GROWTOGETHER**) to continuously operate the records from the data such as XP updates, streak progress, users interact with other users, focus sessions, and achievements, so to handle fast real-time activity, we need a database which can efficiently store items without table structures as the group can create the table of the data manually or in json format. DynamoDB's key model supports the flexibility of the model while securing fast operations which are required for the group project, as the group records the real-time session records, buddy synchronization, gaining experience, and for the streak updates. The serverless characteristic of the model and auto-scaling integrates with our AWS Lambda backend, allowing the app's system to run smoothly as more sessions and users are added to the database.

3.2 Table Schemas

g_users

used in (g_users, users (updateStreak), Users (pings))

Attribute	Purpose
user (PK)	identity
username (SK)	unique identity
achievement	storing the user achievements
buddy_id	buddy id info
buddy_link	buddy connection
buddy_link_at	timestamp when the buddy link was created
focus_today	focused today
hours	accumulated hours
streak_count	daily streak for hitting the goals
xp	experience points
email	contact
joined_at	user account when created
last_ping_at	for reminders
last_session	recent session
password	authentication
sessions	list of sessions
user_id	UUID ID for the user
working_on	active session/task

Data Structure for g_users (JSON format)

```
{  
  "user": { "S": "1"},  
  "username": { "S": "kasmot"},  
  "user_id": { "S": "0001"},  
  "email": { "S": "enr.kytepercia@gmail.com"},  
}
```

```

"password": { "S": "Kasmot042803+"},
"buddy_link": { "S": "78SD4ajT"},
"buddy_id": { "S": "0002"},
"buddy_link_at": { "S": "04/28/2003"},
"xp": { "N": "55"},
"focus_today": { "BOOL": false},
"hours": { "L": []},
"achievement": { "L": []},
"joined_at": { "S": "2025-11-14"},
"last_session": { "S": "2025-11-26"},
"streak_count": { "S": "2"}
"working_on": { "S": { "session" : { "S" : "focus" }, "reps" : { "N" : "3" } }}
}

```

g_partnerStats

used in g_partnerStats

Attribute	Purpose
user (PK)	identity
buddy_id (SK)	paired buddy username
combinedHours	total hours of the both users
daysBothHit2Hours	count of days for the both of the users in reaching 2 hours (together)
matchingStreakDays	streak of consecutive days
pingsSent	for reminders
reliableResponses	number of times the buddy responded reliably
totalSyncHours	total hours spent (same-planned sessions)
updatedAt	recent time when the stats of the partner were updated
username	unique identity

Data Structure for g_partnerStats (JSON format)

```

{
  "user": { "S": "user"},
  "buddy_id": { "S": "96a06be0-9c09-4ce5-82fb-3825b75de656"},
  "combinedHours": { "N": "1.25"},

```

```

"daysBothHit2Hours": { "N": "0"},
"matchingStreakDays": { "N": "0"},
"pingsSent": { "N": "1"},
"reliableResponses": { "N": "0"},
"totalSyncHours": { "N": "0"},
"updatedAt": { "S": "2025-11-28T21:15:38.416705+00:00"},
"username": { "S": "[ { "S" : "Test User" }, { "S" : "asmond" } ]"},
}

```

g_userStats

used in g_userStats

Attribute	Purpose
user (PK)	identity
username (SK)	unique identity
currentStreak	current number streaks of the users
firstSessionAt	timestamp of the first session
lastSessionAt	timestamp of the last session
longestStreak	highest consecutive streak for both users achieved
maxHoursSingleDay	maximum hours of focus hours of the users has logged in a single day
monthlyHours	total accumulated hours in a month
pomodoro1Count	count of short pomodoro session
pomodoro2Count	count of medium pomodoro session
pomodoro3Count	count of long pomodoro session
totalHours	all time total accumulated hours
totalSessions	all time total accumulated sessions
updatedAt	recent time when the stats were updated
weeklyHours	total accumulated hours in a week

Data Structure for g_userStats (JSON format)

```
{
  "user": { "S": "user" },
  "username": { "S": "Test User" },
  "currentStreak": { "N": "0" },
  "firstSessionAt": { "S": "2025-11-26T18:09:55.984796+00:00" },
  "lastSessionAt": { "S": "2025-11-26T19:18:03.612684+00:00" },
  "maxHoursSingleDay": { "N": "0.416666666666666666666666666667" },
  "monthlyHours": { "N": "0.416666666666666666666666666667" },
  "pomodoro1Count": { "N": "2" },
  "pomodoro2Count": { "N": "0" },
  "pomodoro3Count": { "N": "0" },
  "totalHours": { "N": "0.833333333333333333333333333333" },
  "totalSessions": { "N": "1" },
  "updatedAt": { "S": "2025-11-28T20:38:25.743040+00:00" },
  "weeklyHours": { "N": "0.833333333333333333333333333333" },
}
```

g_sessions

used in g_sessions

Attribute	Purpose
session (PK)	session records under the session entity type
session_id (SK)	unique session identifier
completed	boolean to know if the tasks are completed
created_at	timestamp of the records when created
description	deep focus, light focus (type of session)
durationMinutes	number of minutes completed in the session
endTime	timestamp when the session ended
reps	number of type of session completed
session_name	name of the session
startedTime	timestamp when the user started the session
time	elapsed time
timeAllotted	planned time in completing the session

updated_at	timestamp of the most recent update
users	list of the users

Data Structure for g_sessions (JSON format)

```
{
  "session": { "S": "session" },
  "session_id": { "S": "270266f8-ad8f-4690-b78d-90123c3cea22" },
  "completed": { "BOOL": "false" },
  "created_at": { "S": "2025-11-26T20:48:37.256670+00:00" },
  "description": { "S": "Kay super man" },
  "durationMinutes": { "N": "0" },
  "endTime": { "S": "null" },
  "reps": { "N": "0" },
  "session_name": { "S": "I want to" },
  "startedTime": { "S": "2025-11-26T21:01:01.848841+00:00" },
  "time": { "S": "25-5" },
  "timeAllotted": { "S": [] },
  "updated_at": { "S": "2025-11-26T21:01:01.848841+00:00" },
  "users": { "S": "[ { \"S\" : \"asmond\" }, { \"S\" : \"Test User\" } ]" }
}
```

Users (Pings)

used in Pings

Attribute	Purpose
user_id (PK)	identity
buddy_id	buddy id info
email	buddy contact
nickname	buddy name
last_ping_at	last ping sent

Data Structure for Users (Pings) (JSON format)

```
{
  "user_id": { "S": "user-A" },
  "buddy_id": { "S": "user-B" },
  "email": { "S": "john@gmail.com" },
  "last_ping_at": { "S": "1764071743" },
}
```

```
"nickname": { "S": "John"},
}
```

Forest

used in: getForestState

Attribute	Purpose
forest_id (PK)	unique forest
user_a_id	first user
user_b_id	second user
xp_total	combined XP
trees	number of trees
last_updated	timestamp
level	forest level

Data Structure for forest (JSON format)

```
{
  "forest_id": { "S": "user-A"},
  "user_a_id": { "S": "user-B"},
  "user_b_id": { "S": "john@gmail.com"},
  "xp_total": { "N": "1764071743"},
  "trees": { "N": "John"},
  "last_updated": { "S": "John"},
  "level": { "N": "John"},
}
```

g_achievementUpdate

used in g_achievementUpdate

Attribute	Purpose
user (PK)	identity
achievement_id (SK)	unique id for a specific achievement
accountability	tracks achievements related to accountability behavior

combined	combined achievements of the buddy
milestone	milestone
partnerDuration	achievements in regards to the user who stayed with the same buddy
pomodoro	achievements for completed pomodoro
powerUser	power user style achievements
streak	streak-based achievements
sync	achievements for synchronous activity
volume	high-volume achievements

Data Structure for g_achievementUpdate (JSON format)

```
{
  "user": { "S": "user" },
  "achievement_id": { "L": [] },
  "accountability": { "L": [] },
  "combined": { "L": [] },
  "milestone": { "L": [] },
  "partnerDuration": { "L": [] },
  "pomodoro": { "L": [] },
  "powerUser": { "L": [] },
  "streak": { "L": [] },
  "sync": { "L": [] },
  "volume": { "L": [] },
}
```

g_xpStats

Attribute	Purpose
user (PK)	identity
username (SK)	unique identity
buddyHoursCounted	total number of combined focus hours
buddyXp	total exp earned from the user's buddy
level	optional
soloHoursCounted	total number of solo focus hours
soloXp	total exp earned from solo activity

totalXp	total exp of the user
updatedAT	timestamp of the latest exp
xpNextLevel	optional

Data Structure for g_xpStats (JSON format)

```
{
  "user": { "S": "user" },
  "username": { "S": "Test User" },
  "buddyHoursCounted": { "N": "2" },
  "buddyXp": { "N": "4" },
  "level": { "N": "5" },
  "soloHoursCounted": { "N": "12" },
  "soloXp": { "N": "2" },
  "totalXp": { "N": "52" },
  "updatedAT": { "S": "asdsadasdsadas" },
  "xpToNextLevel": { "N": "4" },
}
```

3.3 Data Relationships

The project **GROWTOGETHER** application, is modeled through references across various DynamoDB tables. The main identity of the table is stored in **g_users** (profile of the users), where each of the users has a unique username identification and **buddy_id**, and real-time updates from the **streak-count**, **xp**, **hours**, and **working on**. Aside from the main identity, there are partner-specific metrics which can be seen in **g_partnerStats**, where the user and **buddy_id** capture data from other tables, for instance, the **totalSyncHours**, **combinedHours**, and **pingsSent**, while the individual collects **totalHours**, **weeklyHours**, and the **pomodoroCount** which is stored in **g_userStats** keyed by (**user** and **username**). Every focus in the database record is stored in **g_sessions**, and the users field in each session links back to one or more usernames from **g_users**, making it viable in deriving both buddy and solo statistics. The **Pings** (Users), reflect the buddy link, to keep track of the **buddy_id**, **user_id**, and **last_ping_at**. The progress of the users is also documented in **g_xpStats**, which summarizes the experiences of the solo and **buddyXp** per user. The forest table collects the combined experience of the users for a paired forest state (**xp_total**, **level**, **trees**), and lastly, the **g_achievementUpdate**, as this table tracks the achievements of each user unlocked across categories such as **sync**, **pomodoro**, **streak**, and **accountability**. Altogether, the references from a coherent flow of tables and relationships where sessions, buddies, users, and forests stay consistent through references.

3.4 Access Patterns

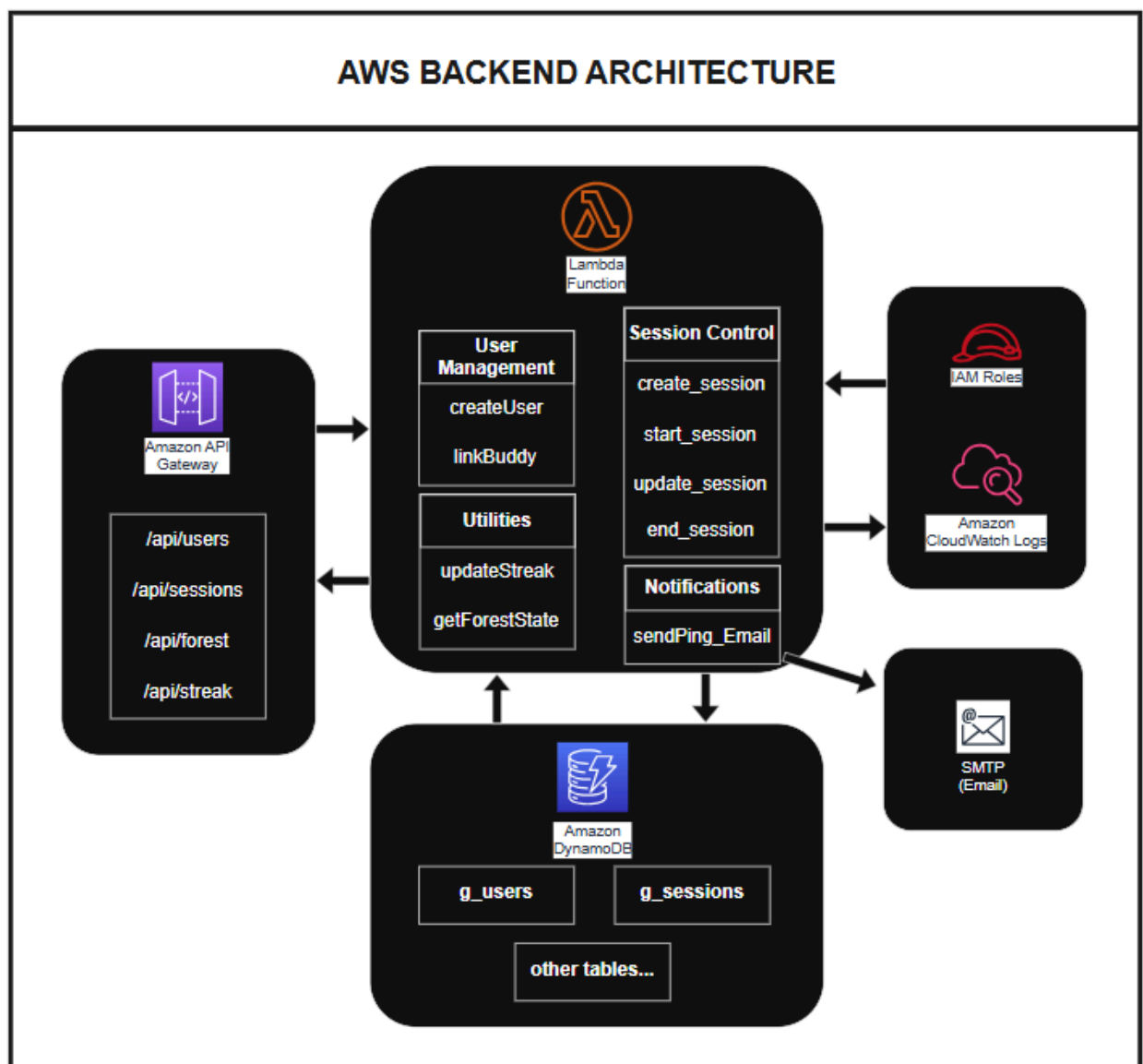
The group application, which is **GROWTOGETHER**, is designed around predictable access which is optimized for DynamoDB. The system of the application retrieves user profiles from the main table, which is **g_users** using the user (**partition key**) and username (**sort key**) to fetch significant data like **experience**, **buddy links**, **progress of the streak**, and **session date**. The session-related operations are usually retrieved or updating the focus session in the table

g_sessions by querying with its sort key (**session_id**), to enable real-time tracking of the completion of the session, the duration of the session, and the lists of the participants. The analytics are accessed through **g_partnerStats**, where queries of the application (**users**, **buddy_id**) obtain the combined hours, sync, and accountability metrics. And as the group stated earlier, long-term individual metrics are accessed directly from the **g_userStats**, while the progression of XP is gained through simple lookups in **g_xpStats**. The system reminder, which is **Pings**, determines the user's last ping to their buddy, and also, the progression of the shared system queries the forest table using **forest_id** to have some experience and forest level. These patterns allow the application to operate real-time updates and lookups efficiently while keeping the DynamoDB usage scalable and cost-effective.

4. Backend

4.1 Cloud Architecture Overview

The backend is composed mainly of API calls to call modular functions utilizing different services of Amazon Cloud Services interacting with one another.



4.2 Dependencies

All functions were written in Python **3.14** with the following dependencies:

Package	Description	Version
boto3	AWS SDK for Python - enables Python developers to write software that uses AWS services	1.41.5 (latest as of Nov 2024)
json	JSON encoder and decoder for Python	Built-in (Python Standard Library)
decimal	Decimal fixed-point and floating-point arithmetic	Built-in (Python Standard Library)
time	Time access and conversions	Built-in (Python Standard Library)
smtplib	SMTP protocol client for sending email	Built-in (Python Standard Library)

4.3 Environment Configuration

Environment Variable	Description	Default Value	Used By Functions
SESSION_TABLE_NAME	DynamoDB table for session data	g_sessions	create_session, start_session, update_session, end_session, logSession
USER_TABLE_NAME	DynamoDB table for user data	g_users	create_session, createUser, linkBuddy, logSession
USERS_TABLE	DynamoDB table for user data	(none)	sendPing_Email

COOLDOWN_SECONDS	Ping cooldown period in seconds	7200	sendPing_Email
SMTP_EMAIL	Email address for SMTP sender	<i>(required)</i>	sendPing_Email
SMTP_PASSWORD	Password for SMTP authentication	<i>(required)</i>	sendPing_Email

4.4 Lambda Functions

4.4.1 *create_session*

Creates a new study session between two users and appends the session_id to both users' session lists.

POST `api/sessions/create`

Parameters

```
{
  "user_id": "string (required)",
  "partner_id": "string (required)",
  "mode": "string (required)",
  "session_name": "string (optional)",
  "description": "string (optional)"
}
```

Return

```
{
  "statusCode": 201,
  "headers": {"Content-Type": "application/json"},
  "body": {
    "message": "Session created.",
    "session": {
      "session": "session",
      "session_id": "uuid",
      "session_name": "string",
      "users": ["user_id", "partner_id"],
      "time": "mode",
      "created_at": "ISO timestamp",
      "updated_at": "ISO timestamp",
      "description": "string",
      "reps": 0,
      "completed": false,
      "durationMinutes": 0,
      "timeAllotted": []
    }
  }
}
```

4.4.2 *start_session*

Marks a session as started by setting the `startedTime` timestamp. Prevents starting an already-started session.

POST `api/sessions/start`

Parameters

```
{
  "session_id": "string (required)"
}
```

Return

```
{
  "statusCode": 200,
  "headers": {"Content-Type": "application/json"},
  "body": {
    "message": "Session started.",
    "session": {
      "...session_attributes",
      "startedTime": "ISO timestamp"
    }
  }
}
```

4.4.3 *update_session*

Updates session metrics by incrementing duration, reps, and time allocations. Uses delta values to add to existing totals.

POST `api/sessions/update`

Parameters

```
{
  "session_id": "string (required)",

```

```
"duration_delta": "integer (optional, default: 0)",  
"reps_delta": "integer (optional, default: 0)",  
"date": "string YYYY-MM-DD (optional)",  
"time_a_delta": "integer (optional, default: 0)",  
"time_b_delta": "integer (optional, default: 0)"  
}
```

Return

```
{  
  "statusCode": 200,  
  "headers": {"Content-Type": "application/json"},  
  "body": {  
    "message": "Session updated.",  
    "session": {  
      "...session_attributes",  
      "durationMinutes": "updated_value",  
      "reps": "updated_value"  
    }  
  }  
}
```

4.4.4 *end_session*

Marks a session as ended and completed by setting endTime timestamp and completed flag to true.

POST `api/sessions/end`

Parameters

```
{
  "session_id": "string (required)",
}
```

Return

```
{
  "statusCode": 200,
  "headers": {"Content-Type": "application/json"},
  "body": {
    "message": "Session ended.",
    "session": {
      "...session_attributes",
      "endTime": "ISO timestamp",
      "completed": true
    }
  }
}
```

4.4.5 *sendPing_Email*

Sends an email notification to a user's study buddy with a 2-hour cooldown period. Uses SMTP with HTML email template.

POST `api/sessions/sendPingEmail`

Parameters


```
{
  "username": "string (required)"
}
```

Return

```
{
  "statusCode": 200,
  "body": {
    "success": true,
    "channel": "email"
  }
}
```

4.4.6 *createUser*

Creates a new user account with generated UUID and buddy link.

POST `api/users/create`

Parameters

```
{
  "name": "string (required)",
  "email": "string (required)",
  "nickname": "string (required)",
  "password": "string (required)"
}
```

Return

```
{
  "statusCode": 201,
  "headers": {"Content-Type": "application/json"},
  "body": {
    "message": "User created successfully.",
    "user": {
      "user": "user",
      "user_id": "uuid",
      "username": "name",
      "email": "email",
      "password": "hash",
      "buddy_link": "uuid",
      "buddy_id": null,
      "xp": 0,
      "focus_today": false,
      "achievement": [],
      "joined_at": "ISO timestamp",
      "last_session": "ISO timestamp",
      "streak_count": 0
    }
  }
}
```

4.4.7 *linkBuddy*

Links two users as study buddies using either `buddy_link` or `buddy_code`. Both users must not have existing partners.

POST `api/users/link`

Parameters

```
{
  "username": "string (required)",
  "buddy_link": "string (optional - one required)",
  "buddy_code": "string (optional - one required)"
}
```

Return

```
{
  "statusCode": 200,
  "headers": {"Content-Type": "application/json"},
  "body": {
    "message": "Buddy linked successfully"
  }
}
```

4.4.8 *login*

Login lambda function.

POST `api/users/login`

Parameters

```
{
  "username": "string (required)",
  "password": "string (required)",
  "buddy_code": "string (required)"
}
```

```
}
```

Return

```
{
  "statusCode": 200,
  "headers": {"Content-Type": "application/json"},
  "body": {
    "message": "Buddy linked successfully"
  }
}
```

4.4.9 *updateStreak*

Updates user's streak count based on last session date.

POST `api/streak/update`

Parameters

```
{
  "username": "string (required)"
}
```

Return

```
{
  "statusCode": 200,
  "headers": {
    "Content-Type": "application/json",
    "Access-Control-Allow-Origin": "*"
  },
  "body": {
    "message": "Streak updated",
    "streak_count": "integer"
  }
}
```

4.4.10 *getForestState*

Retrieves forest item stats and its associated users' stats (XP, streak) for a partnership.

Get `api/forest/state`

(Query) Parameters

`?forest_id=string (required)`

Return

```
{
  "statusCode": 200,
  "headers": {
    "Content-Type": "application/json",
    "Access-Control-Allow-Origin": "*"
  },
  "body": {
    "forest_id": "string",
    "level": "integer",
    "xp_total": "integer",
    "last_updated": "integer",
    "trees": "integer",
    "user_a": {
      "user_id": "string",
      "xp": "integer",
      "streak": "integer"
    },
    "user_b": {
      "user_id": "string",
      "xp": "integer",
      "streak": "integer"
    }
  }
}
```

4.4.11 *get_buddy_info*

Retrieves a user's buddy information based on their username (referred to as `buddy_link`).

GET `api/info/buddy`

(Query) Parameters

```
{
  "buddy_link": "string"
}
```

Return

```
{
  "statusCode": 200,
  "headers": {
    "Content-Type": "application/json",
    "Access-Control-Allow-Origin": "*"
  },
  "body": {
    "message": "Buddy fetched successfully",
    "buddy": {
      "user": "user", // Partition Key value
      "username": "string", // Username of the buddy
      "full_name": "string", // Full name of the buddy
      "email": "string", // Email of the buddy
      "xp": "integer", // XP value
      "streak": "integer", // Streak value
      // Other buddy attributes
    }
  }
}
```

```
}
```

4.5 Other Details

4.5.1 Authentication for API Calls

Basic Authentication (*username, password*) is required for every API call. This is to prevent calls outside of the application.

Note that Basic Authentication is just a temporary fix for MVP version.

4.5.2 IAM Permissions Required

Function	DynamoDB Actions	Other AWS Services
create_session	dynamodb:PutItem, dynamodb:GetItem, dynamodb:UpdateItem on g_sessions and g_users	None
sendPing_Email	dynamodb:GetItem, dynamodb:UpdateItem on USERS table	None (uses external SMTP)
createUser	dynamodb:PutItem on g_users	None
All functions	logs:CreateLogGroup, logs:CreateLogStream, logs:PutLogEvents	CloudWatch Logs

4.5.3 Rate Limits

Function	Limit	Window	Notes
sendPing_Email	1 request per user	2 hours (7200s)	Cooldown enforced in code
create_session	10 requests per user	1 minute	TO BE IMPLEMENTED

All endpoints	100 requests per IP	1 minute	API Gateway throttling
---------------	---------------------	----------	------------------------

4.5.4 Business Logic Rules

Rule	Description	Affected Functions
Unique Buddy Pairing	Each user can only have one buddy at a time	linkBuddy, createUser
Session Lifecycle	Sessions must progress: created → started → ended	create_session, start_session, end_session
Streak Calculation	Streak increments if last_session was yesterday (Philippine time)	updateStreak
XP Accumulation	XP earned from completed sessions only	end_session, getForestState
Ping Cooldown	Minimum 2 hours between pings to same buddy	sendPing_Email

4.5.5 Dependencies Between Functions

User Registration Flow:

createUser → [User can now] → linkBuddy → [Ready for sessions]

Study Session Flow:

create_session → start_session → update_session (multiple) → end_session

Notification Flow:

sendPing_Email (triggered by user action or schedule)

Progress Tracking:

updateStreak (scheduled daily or on session completion)

getForestState (called by frontend for display)

5. Project References

5.1 Flutter Documentation

Resource	URL
Flutter Official Docs	https://docs.flutter.dev
Flutter HTTP Package	https://pub.dev/packages/http
Provider Package	https://pub.dev/packages/provider
FL Chart Package	https://pub.dev/packages/fl_chart

5.2 AWS Documentation

Resource	URL
AWS Lambda	https://docs.aws.amazon.com/lambda
AWS DynamoDB	https://docs.aws.amazon.com/dynamodb
AWS API Gateway	https://docs.aws.amazon.com/apigateway
Boto3 (Python SDK)	https://boto3.amazonaws.com/v1/documentation/api/latest/index.html

5.3 Glossary of Terms

Term	Definition
API	Application Programming Interface allows apps to communicate
Lambda	AWS serverless compute service that runs code without managing servers
DynamoDB	AWS managed NoSQL database service
NoSQL	Non-relational database with flexible schema (no fixed tables/columns)
API Gateway	AWS service that acts as "front door" for API requests
IAM	Identity and Access Management - controls AWS permissions
CloudWatch	AWS monitoring and logging service
SMTP	Simple Mail Transfer Protocol is a standard for sending emails
Pomodoro	Time management technique using a defined duration focus intervals with rests in between
XP	Experience Points or the gamification metric for user progress
UUID	Universally Unique Identifier is random ID for records
SDK	Software Development Kit is tools for building software
Serverless	Architecture where cloud provider manages servers automatically
Boto3	AWS SDK for Python
Flutter	Google's UI toolkit for cross-platform app development
Dart	Programming language used by Flutter

State Management	How an app tracks and updates data across screens
Widget	Basic building block of Flutter UI
Streak	Consecutive days of completed focus sessions
Buddy System	Feature pairing two users for accountability

6. Appendices

6.1 Role Descriptions

Role	Description
Project Manager	Oversees the overall development of the application and ensures that timeline and project goals will be met.
Front-end Developer (Mockup Focused)	Design the mockup of the mobile/web user interface.
Front-end Developer (Flutter Focused)	Implements the mobile/web user interface and also ensures usability, responsiveness, and integration with backend APIs
Back-End Developer	Implement server-side logic, and APIs and also manage AWS Lambda functions, DynamoDB tables, API Gateway, and SNS for notifications. <i>This includes the necessary documentation.</i>
Database Manager	Designs and maintains database schema, data flow, and storage efficiency.
QA Specialist	Evaluates the competency of the application (like finding bugs or specifying edge cases) and gives feedback and suggestions for improvements.
Documentation	Prepares the technical documentation of the overall development of the application.

6.2 Roles & Contributions

Role	Name	Contributions
Project Manager/Back-End	Kyte Daiter Percia	Management of the group and delegations. achievementUpdate
Front-end Developer (Mockup Focused)	Jade Marco Morillo	Mockup of UI/UX using Canva
Front-end Developer (Flutter Focused)	Jan Edward Macagba	Flutter implementation and API integration
Back-End	Jhon Khriel Cymon Empeno	createUser, linkBuddy, logSession
Back-End	Darlyn Antoinette Baybayon	updateStreak, getForestState
Back-End	Lindsay Faith Bazar	sendPing
Database Manager	John Carlos Quintero	Creation and management of database
QA Specialist / Documentation	Romand Lansngan	Achievements Lists Documentation Debugging and Improvements API

6.3 Github Link

The repository for ***GrowTogether*** can be accessed through this link:
<https://github.com/Jemacsss/feu-mobilecloud-finalproj>